Layered Agent Authorization with Exact Grants, Cited Rules, and Escalation

OpenBot

***Abstract*—Agent systems must decide which actions may run without asking a person each time. We define a layered policy with exact Grants, plain language Rules, an abstaining judge, and an owner chain. A real Rule quote proves that text exists, but need not support an action. Our method checks complete Rule evidence before a decision. It separates decision risk from scope risk and gives a finite-sample procedure for choosing allow and deny thresholds. We prove a conditional soundness result and state the extra conditions needed for safe reuse. A finite experiment checks all 14,425 label-score lists of length at most three against a separate aggregation oracle. Every result agrees; this tests the selection logic, not the truth of model labels. We also give negative controls, workload equations, and a held-out study protocol. No learned judge or human trial is reported. The results establish bounded protocol properties under an assumed deterministic authority boundary.**

***Index Terms*—Agent authorization, Selective prediction, Human oversight, Policy decisions**

I. Introduction

An agent may have the right to use a mail service without having the right, in the person's intended sense, to send every message it can form. The first right can be tested against a fixed access policy. The second depends on the action, its arguments, and instructions such as "send the weekly report to the team, but ask before sending it outside." A useful system must decide some such requests itself. It must also know when to ask.

Repeated requests can use more attention than the task itself. Trusting a language model moves that cost into uncertain actions. Broad permission creates a further risk: an old answer can cover a later case that the person did not intend. Approval load is tied to decision error and reuse scope.

We define a layered decision system with four stages: an authority check, stored Grants, plain language Rules, and a chain of deciders. A Grant stores an allow or deny. A Rule is text that a judge model reads. The judge can answer `no_match`, which means that it does not decide. A pending request can move through agent owners to a person. The requesting agent can continue other work while the action waits.

This paper studies decision quality inside that structure. It assumes a deterministic boundary that checks the current actor and token before an outside action. An allow inside our policy cannot create a right that this boundary withholds. Correct binding of a decision to its action is also assumed; our claims concern the content of that decision. An action can pass that boundary and still violate a person's Rule. Conversely, denying all such actions would preserve the boundary while making the agent of little use.

We separate text provenance, meaning, authority, and reuse through an explicit baseline. We then define a selective method with allow risk, deny risk, and review cost. Selective means that a model may withhold a decision. Worked traces and an evaluation plan address repeated errors and human attention. The contributions are a decision rule with explicit proof conditions, a risk-control procedure that accounts for threshold search, and a reproducible finite experiment with negative controls. The experiment tests small models of the protocol. It does not establish a gain in language-model accuracy, attack resistance, or human attention.

II. Problem and Assumptions

A request contains an agent, an action, a target, optional arguments, a conversation, and the agent's stated reason. A capability is the action, target, and arguments together. A decision has two parts: its effect, allow or deny, and its reuse scope. The available scopes are one call, one conversation, and lifetime until revocation. "Lifetime" is a storage choice, not a claim that circumstances never change.

Let a request be x, the active Rule set be R, and the reference judgment be y. The reference has three values: allow, deny, and unresolved. Unresolved means that the available policy does not settle the case. It is distinct from a denied action. A model that allows an unresolved request has made an unsupported decision even if a person later chooses to allow it. This distinction prevents evaluation from rewarding lucky guesses about missing intent.

We assume that the reference panel can see the complete capability and policy available at decision time. Missing facts remain missing. The agent's reason can explain a goal, but it cannot establish facts such as who owns a destination. The action's schema must expose the fields that matter to the Rule. If a tool has hidden effects, correct text reasoning cannot recover them from an incomplete request.

The threat model permits an attacker to influence the request's reason or string arguments. The attacker may place instructions in content that the acting agent later copies. It cannot directly change the member policy or trusted Rule store. A trusted Rule may still be vague or wrong. An agent reviewer may make the same language error as the first judge. A person may be distracted. We therefore treat review as a fallible decision stage with a cost, not an oracle.

A. Decision State and Trust Boundary

For a fixed policy snapshot, write the true effect of Rule i as z_i in $\{A, D, I, U\}$. The labels mean allow, deny, irrelevant, and unresolved. A Rule includes its own exceptions. Thus

"send to the team, except private reports" is one condition whose exception must be checked. Define the reference effect Y as deny when any $z_i = D$; otherwise it is allow when at least one $z_i = A$ and every z_i is A or I. All other cases are unresolved. This is the paper's chosen combination policy. It is not a claim that all organizations intend deny precedence.

The active Rule list, its versions, the capability, and the facts used in the decision form one snapshot. The authority layer must bind that snapshot to the admitted call or reject a stale result. A changed destination, changed Rule, or replaced token needs a new check. Our soundness result concerns one such snapshot. It does not make a decision made before revocation valid after revocation.

The trusted part consists of the Rule store, complete Rule enumeration, typed fact readers, the state store, and the authority check. The model and support checker are fallible. A hash records which inputs were checked; it does not make their meaning true. A crash before a saved decision gives no allow. A failed policy read stops the call. A model deadline yields no decision. These cases must have distinct audit outcomes so an unavailable checker cannot appear as a deliberate denial.

The model permits an administrator to mark a class of calls implicit, or to require direct human review. The main study excludes these routes from its judge denominator. We record their volume separately, since including them in judge coverage would confuse administrative choices with model ability. The study also separates failures to read policy from uncertainty about a Rule. The former stops the call; the latter may create a review request.

III. Related Work

Selective classification gives a model a reject option and studies the relation between coverage and risk. Geifman and El-Yaniv use selection to control risk for a trained neural classifier [1]. Our setting has a similar choice to abstain, but its output changes an action and may persist as a Grant. A single classification error and a repeated error caused by one reused decision must therefore be counted separately. Their image classification results do not supply an error bound for this application.

Learning to defer goes further by modeling the downstream expert. Mozannar and Sontag study joint prediction and deferral with a consistent surrogate loss [2]. This is closer to our workload objective than treating every rejected request as equally expensive. Our model adds a chain of possible reviewers and a requirement that an agent reviewer hold the capability before its agreement can finish the request. Reviewer skill and reviewer authority are distinct variables.

Calibration concerns whether reported confidence matches observed correctness. Guo and colleagues show that neural confidence can be poorly calibrated and study post-processing methods [3]. The decision-model baseline defined below uses a probability of at least 0.5 to select a Rule for citation. That number is not a measured probability that the final authorization is correct. Our proposal calibrates final decision errors on separate data and keeps allow and deny results apart.

Human oversight also needs direct measurement. Bansal and colleagues found that explanations did not improve complementary team performance in their study and could increase acceptance of incorrect advice [4]. We do not transfer their effect size or task outcome to authorization. The finding motivates a comparison between plain request data and request data with model explanations. A polished reason and a correct-looking quote may raise trust without raising correctness.

Greshake and colleagues show how instructions placed in retrieved data can affect applications built around language models [5]. Wallace and colleagues study training models to respect instruction priority [6]. Both inform our treatment of request text as untrusted data. They address a cause of judge error, but neither makes substring citation checking a proof of meaning. Our specific research question is how much trusted decision coverage remains after support, conflict, and uncertainty checks, and how much review work that coverage saves.

ALCE measures answer correctness and citation quality as separate parts of cited text generation [7]. Our quote counterexample is an authorization instance of this established distinction. It adds action arguments, conflicting Rules, and later reuse; the general fact that a citation can fail to support a claim is not new.

Risk control must also account for choosing a threshold after seeing data. Learn then Test casts parameter selection as a family of statistical tests [8]. We use a simple instance: separate binomial bounds for allow and deny over a fixed finite grid. This statistical construction is established work. Its use here makes the review policy and its sampling unit explicit; it does not turn model confidence into a proof for each action.

Macaroons support delegated credentials whose added conditions restrict use [9]. The same restriction principle motivates our scope test. We add no new cryptographic credential. Our question is when an uncertain language decision may justify a reusable Grant, and how to count the errors caused by that reuse. State-based one-call limits and current policy checks remain separate obligations.

IV. A Defined Baseline Policy

This abstract baseline makes no claim about a deployed product. All evaluated methods share its authority boundary and request format. Differences concern Rule decisions, review, and reuse.

A. Exact Grant Matching

A Grant stores an agent ID, action, target constraint, optional argument constraint, effect, scope, conversation ID when needed, status, giver, and optional parent Grant. Status is active, used, or revoked. Match a request only when the agent and action agree, the Grant is active, and its scope includes the request. Conversation scope matches only its named conversation. Lifetime scope has no time limit and lasts until revocation. Only allow Grants may have one-call scope. Such a Grant can be claimed once by an atomic change from active to used.

The target constraint is either a literal string or `*`, which accepts every target. When an argument constraint is present, compare its serialized JSON bytes with the request's serialized argument bytes. Missing request arguments serialize as `{}`. An omitted Grant constraint accepts any arguments. Serialization preserves input key order in this baseline. It follows that two objects with the same fields in different orders can fail to match; no semantic JSON equality is claimed.

Collect every matching Grant. If none match, return a miss. If any has effect deny, return deny. Otherwise choose the first allow in fixed creation order. A reusable allow returns allow. For a one-call allow, attempt the atomic claim; if it fails, proceed as a Grant miss. Thus deny precedence is deterministic within this stage. A wide target or omitted arguments broadens the set of matching actions. "Exact" means equality of specified constraints, not that every Grant fixes a complete call.

The stages use first-decision order. An allowed Grant returns before Rules are read. A natural language deny therefore does not globally override an earlier Grant. This is an explicit policy choice. A system that needs global deny precedence must check those prohibitions before returning a Grant result. Changing that order is outside the main comparison because it would change which actions reach the judge.

B. A Substring-Citation Judge

On a Grant miss, the baseline sends the agent ID, name, role, capability, reason, and all active Rules to a judge. Each Rule contains an ID, version, author, and text. The prompt asks for `allow`, `deny`, or `no_match`; it labels request text as data and asks for a clear Rule basis. The output also contains an explanation and zero or more pairs of Rule ID and quoted text.

The post-check trims each quote. It retains the citation exactly when the ID names an active Rule, the quote is nonempty, and the quote occurs as a substring of that Rule. If an allow or deny has no retained citation, return `no_match`. Otherwise keep the model's label and retained citations. An explicit `no_match` stays undecided. The algorithm does not test whether a quoted span supports the chosen action. It also does not aggregate independent Rule labels or search for an omitted conflicting Rule.

Define a second baseline with a constrained decision model. It chooses the same three labels and supplies one relevance probability for each Rule. Quote the entire text of each Rule with probability at least 0.5, then apply the same post-check. This limits free-form output but leaves meaning to the model. A relevance score is not a measured probability of a correct final authorization.

Both baselines return `no_match` on malformed output, model failure, or a fixed 15-second deadline. Cancellation stops the attempt and produces no verdict. A cache experiment uses a deliberately narrow key: agent ID, ordered active Rule IDs and versions, and serialized capability. It omits reason, name, role, and model configuration. This fully defined variant tests whether reuse silently ignores prompt inputs that affect decisions. The proposed method instead binds a cached result to all decision inputs and checker versions, or removes an input from the prompt before declaring it irrelevant.

C. Review States and Scope

A miss creates a pending request containing the complete call, current decider, owner chain, revision, and deadline. Pending does not run the action. The agent may do other work while review proceeds. The owner chain is finite and has no repeated agent. It ends at a person and can differ from the path by which the task arrived.

A reviewer may deny, allow, or escalate. An agent's allow can finish only if its separate holding check confirms the capability. Otherwise its allow becomes an endorsement, with a chosen scope, and the request moves to the next owner. Escalation without endorsement also moves upward. Every change increments the revision. A reply must name the current decider and revision; a stale reply cannot replace the saved state. If no decision arrives by the deadline, the request expires without running. This paper studies no default-allow deadline.

When a final allow arrives, construct Grants down the endorsement path, or directly to the requester when that path is empty. The final decider grants to the highest endorsing agent, which grants to the next, until the requesting agent receives its Grant. Each giver chooses the scope at its own link. Each lower Grant records the Grant above as parent. A one-call Grant is marked used for the waiting call. A denial with conversation or lifetime scope creates a deny Grant for the requester. A one-call denial leaves no reusable Grant.

A person can also author a Rule. For an agent-authored allowing Rule, require its author to be above the requester and to hold the capability. Holding is a read-only check. It applies the Grant matching and deny-precedence rules but does not claim or consume a one-call Grant. On a Grant miss, it may use an allowing Rule decision based only on person-authored Rules. This avoids recursive reliance on agent-authored permission. A positive holding result describes that snapshot; it is not an admission permit and cannot reserve authority by itself.

A review may wait while its supporting Grant changes or is used elsewhere. The final admission must check the support again. If an implementation reserves a one-call Grant during review, bind the reservation to the immutable capability, request ID, review revision, and eventual call ID. The Grant is then unavailable to every other call. Only the bound call can consume that reservation. A stale reply cannot transfer it to another request. The restricted scope variant below gives the same-call rules for a chain of Grants.

These checks constrain who may decide. They do not establish that a language decision is correct. Parent links likewise record provenance but do not prove that every chosen child scope fits within its parent's scope.

V. A Selective Decision Method

The proposed method adds an evidence record before accepting a Rule verdict. The record travels with a review and states the basis for an automatic action.

A. Separate Evidence from the Request

Record the capability, Rule versions, candidate effect, cited spans, and relevant conditions. Each condition names a field as known, absent, or disputed. A reason cannot establish that a destination belongs to the team. Unknown decisive conditions lead to review unless the Rule covers unknown values.

Request strings remain data throughout this step. A copied message that claims to be a new system instruction has no policy status. The judge may report that claim as part of the request, but cannot use it as a Rule. The same restriction applies to an agent's statement that someone already approved. Approval is read from a stored decision, not inferred from persuasive prose.

B. Check Support and Conflict

Evaluate each active Rule against the complete request. For each Rule, produce a label from allow, deny, irrelevant, or unknown; a quote; a list of required facts; and a support score. Unknown includes missing decisive facts, an uncertain interpretation, a failed check, and a Rule omitted by a retrieval limit. An irrelevant label means that the Rule has no effect on this request. Read the full Rule when checking its exceptions, even if the record displays a shorter quote.

A support checker tests whether the label follows from the full Rule and known facts. For a typed condition, such as membership in a named target set, use a direct comparison. For text conditions, use a separately measured checker. A valid substring is required for allow and deny evidence, but is only an anchor to the Rule. It does not settle support. Support scores concern the whole Rule judgment, including relevant exceptions; they are not products of assumed-independent word probabilities.

Aggregate evidence with deny precedence inside the Rule stage. A sufficiently supported deny can decide because another allowing Rule cannot override it under this defined combination policy. Without such a deny, any remaining deny label, unknown label, or low-confidence Rule result forces review. A deny below the decision threshold is still a reason to abstain; it cannot become evidence for allow. Allow requires at least one supported allowing Rule and adequate evidence that every other Rule either allows or is irrelevant. A failed attempt to rule out a denial cannot count as absence of a denial. Listing 1 gives the selection procedure.

```
select(request, active_rules):
  evidence = check_every_rule(request, active_rules)
  if any supported_deny(evidence, deny_threshold):
    return deny
  if any deny_label(evidence):
    return no_match
  if any unknown_or_weak(evidence):
    return no_match
  if no supported_allow(evidence):
    return no_match
  score = calibrated_allow_score(evidence)
  if score < allow_threshold:
    return no_match
  return allow
```

Listing 1. Proposed Rule aggregation. Support includes valid citation, known conditions, and a full-Rule check.

Proposition 1 (sound accepted evidence). Fix one request and policy snapshot. Suppose every active Rule appears exactly once in the evidence, and each accepted allow, deny, or irrelevant label equals its true label z_i. The selection procedure returns only a reference allow or a reference deny. It may still defer a settled case.

Proof. A returned deny has a supported deny label. By soundness this Rule has $z_i = D$, which fixes the reference effect. A returned allow has at least one supported allow label, and every other label is supported allow or irrelevant. Completeness of enumeration leaves no unseen Rule. Soundness therefore gives at least one true allow and no true deny or unresolved Rule. This is the definition of reference allow. No claim is made when the procedure defers. □

This proof needs complete Rule enumeration and sound accepted labels. It does not need the checker to decide every Rule. Missing a Rule must produce unknown; marking an unread Rule irrelevant violates the premise. If enumeration drops a true deny, a strong allow can pass every remaining guard. If a checker labels a true deny irrelevant, complete enumeration alone does not help. These are separate negative controls.

A second model is not a proof checker merely because it is separate. Models can share failures, including the same mistaken exception or forged approval. The learned path supplies estimated evidence. Only a restricted typed Rule language with checked fact readers can discharge the soundness premise by direct computation. The next procedure instead controls an average error rate for a specified population. It does not prove each language judgment correct.

C. Calibrate the Selection Rule

Let $f(x)$ be the proposed effect and $g_t(x)$ indicate that the evidence passes threshold t. Increasing the threshold selects fewer cases from a fixed score ranking. Coverage is $C_t = P(g_t(X) = 1)$. Selective risk is

$$R_t = P(f(X) \neq Y \mid g_t(X) = 1). \tag{1}$$

For a concrete scoring rule, let each verifier return a score for its Rule label. Mark any label below a fixed support cutoff as weak. Use the maximum score among supported denies as the raw deny score. Compute the raw allow score only after all guards pass, as the minimum across the remaining allow and irrelevant labels. Fit a monotone calibration map for each final effect on separate labeled data. The pseudocode applies these maps before thresholding. This construction makes no independence assumption; calibration and threshold validation use different splits. As an illustrative boundary case, let the support cutoff be 0.7, a deny score be 0.8, and the deny threshold be 0.9. The deny-label guard returns `no_match`, whatever the allow score or threshold.

Because errors differ in cost, we also report allow risk conditioned on selecting allow, and deny risk conditioned on selecting deny. An unresolved reference counts as an error for either committed effect. We fit any score calibration on held-out policy families, then select separate thresholds for allow and deny. The defined relevance cutoff of 0.5 remains a baseline; it is not reinterpreted as a calibrated safety threshold.

D. Certify a Fixed Choice of Thresholds

Freeze the model, prompts, fact readers, Rule enumeration, score maps, and a grid of K threshold pairs before reading the certification labels. A pair gives one allow threshold and one deny threshold. Use separate data for model fitting and score fitting. For each pair j and effect e, let $n_{j,e}$ count the selected certification cases and $k_{j,e}$ count wrong decisions. Committing on an unresolved reference is wrong.

Set the family error level to δ, the desired decision risk to α_e, and $\beta = \frac{\delta}{2K}$. Define $U(k, n, \beta)$ as the one-sided exact binomial upper limit. Set $U = 1$ when $n = 0$ or $k = n$. Otherwise it is the unique value u in $[0, 1]$ that solves

$$\sum_{l=0}^{k} \binom{n}{l} u^l (1-u)^{n-l} = \beta. \tag{2}$$

Certify pair j only if $U\big(k_{j,e}, n_{j,e}, \beta\big) \le \alpha_e$ for both effects. A route with no selected sample receives no certificate. It may be disabled; it cannot be called certified. Among certified pairs, choose the one with greatest observed coverage under the declared review budget. Break ties by a fixed rule. If none pass, disable automatic Rule decisions and send their cases to review.

Proposition 2 (finite-grid risk control). If certification requests and future requests are independent draws from the same distribution, all candidate pipelines are fixed before certification, and reference labels are correct, then with probability at least $1 - \delta$, every certified pair has allow risk at most α_A and deny risk at most α_D on its selected populations.

Proof. For one fixed pair and effect, conditional on its selected count, errors have the binomial law with that route's risk. The exact upper limit fails to cover that risk with probability at most β. A union bound over the $2K$ pair-effect tests gives total failure probability at most $2K\beta = \delta$. Choosing among the jointly certified pairs cannot create a failure outside this union. The same examples may appear in several pairs; independence between pairs is not needed. □

This is a statistical statement about a fixed population, not an attack guarantee. The distribution includes the policy families, requests, and model randomness. Reused approvals, a changed tool, or an adaptive attacker can break the sampling premise. In particular, many calls under one wrong Grant are not independent validation samples. A certificate for the Rule route also says nothing about the separate Grant route.

For zero errors, $U = 1 - \beta^{\frac{1}{n}}$. A fixed pair, joint confidence 95 percent, and a one-percent risk target for each effect require at least 368 selected examples of each effect with no error. With 10 or 100 fixed threshold pairs, the count rises to 597 or 826 selected examples per effect. The finite runner checks these counts and verifies that one fewer example fails the same bound. A grid search needs more because β is smaller. These are planning counts, not samples collected by this paper. If the selected fraction for one effect is small, the required total corpus can be much larger.

The objective is useful coverage under a risk and review constraint. The procedure does not guarantee the best population coverage. A low mean calibration error cannot replace its route-specific checks. If a high-impact action class needs its own bound, declare that class in advance and include its tests in the error allocation. Do not infer subgroup protection from an overall average.

E. Treat Scope as a Decision

After an effect is chosen, reuse requires its own evidence. One call is the narrow default. Conversation or lifetime scope requires a reason why the decisive facts will remain stable across future matches. The proposal records which fields were fixed and which were left free. A Rule saying "this report" should not silently justify an unrestricted send tool. A conversation has no fixed end, so conversation scope is not a short time limit.

Scope advice does not change the Grant baseline. We define an additional restricted variant for comparison. Let $D(g)$ be the set of capability, conversation, and time contexts accepted by Grant g, before checking its use count. The recipient ID is not part of this set because a parent and child have different recipients. A child may be saved only when $D(g_c) \subseteq D\big(g_p\big)$. Use the baseline's equality constraints and add an optional expiry to this variant. Check each field: a child action must equal the parent action; a literal parent target or argument byte string must remain fixed; a child may replace a wildcard with a literal; and a conversation-bound parent cannot produce a child that matches another conversation. A child time interval must lie within its parent's interval.

Proposition 3 (scope restriction). A finite chain with checked subset relations admits no capability-context outside its root Grant's set, as long as every ancestor is still valid and either available or reserved for this exact call at admission. This follows by transitivity of set inclusion. It does not prove that the root Grant matches the person's intent.

Use counts need another check. Two children can each have a matching set within a one-call parent and still run twice. At final admission, atomically validate the whole chain and consume each available one-call limit once. For a limit already reserved for this call, consume its saved reservation without claiming a second use. Mark new child Grants used for this same call in that transaction. A used Grant may record evidence for its bound admitted call; it cannot match a new call. Checking each child's local used flag is insufficient.

The same transaction must check current ancestor versions, revocation, expiry, and any reservation binding. A read-only holding check cannot replace it. A saved parent ID alone has no such effect. A reservation does not make a later revocation harmless. The finite scope experiment below tests match sets only, not these database operations.

Compare this restricted variant with unrestricted child scope and with one-call-only reuse. Measure later matches that violate the reference intent, unique mistaken approvals, and all affected calls. The restriction can reduce unintended widening after a correct approval. It cannot repair a wrong root approval, hidden tool effects, or a change in facts that the matching language does not represent.

VI. Analysis and Worked Cases

A. What a Quote Can Prove

Consider the Rule "The agent may read reports but must ask before sending them." A judge returns allow for sending a report and quotes "may read reports." The quote passes a substring check. The decision is unsupported because reading and sending differ. The check has established provenance, meaning where the words came from, and nothing more about their use.

This counterexample uses a real span and a wrong action. It needs no fabricated citation or storage bug. Quoting the entire Rule also passes while leaving the decision wrong. Support must be checked against the action and limiting clause. The same failure applies to the whole-Rule decision-model baseline.

TABLE I
Worked distinctions; these are logical cases, not trial outcomes.

Case	Input change	Decision consequence
Wrong meaning	Read permission cited for a send	Quote exists; proposed support check rejects allow
Grant conflict	Both allow and deny match	Defined Grant policy returns deny
Changed target	Exact Grant names another target	Grant misses; Rules or review decide
False approval	Reason claims prior approval	Claim remains data; stored approval is required
Broad reuse	Arguments omitted from Grant	Later arguments match; scope risk must be measured

B. An Escalation Trace

Suppose a person owns Assistant, Assistant owns Programmer, and Programmer owns Scout. Scout asks to send a report through a named service tool with fixed destination and content. The member policy permits the service. No Grant matches. The only Rule permits reading reports, so a correct judge returns `no_match`.

Programmer supports the send for one call but does not hold the capability. Its endorsement moves the request to Assistant. Assistant also supports it for this conversation and lacks the capability. The person then allows the exact request for lifetime reuse. The request closes, and Grants are made downward: Assistant receives the person's lifetime Grant, Programmer receives Assistant's conversation Grant, and Scout receives Programmer's one-call Grant. The last Grant is already marked used for the waiting call.

Agreement does not itself finish a request, and each chain link can have a different scope. Scout may ask again while an upper agent holds a reusable Grant. Human work can fall while agent review remains. A one-call denial also leaves later requests open to review.

C. Error and Workload Accounting

For a fixed evaluation distribution, partition automatic decisions into Grant, Rule, agent-review, and implicit routes. Let p_i be each route's share of all attempts, and e_i its conditional error rate. Their total wrong-decision mass is $\sum_i p_i e_i$. This follows by partitioning the event that an automatic decision is wrong. It is not evidence that the errors are independent. Grant errors include an earlier mistaken approval and later mismatch between retained scope and intent.

For Rule candidates, selecting fewer requests does not logically guarantee lower risk. A confidence score can rank wrong cases highest. A risk bound requires validation of the selected set under stated sampling assumptions. Let k be the number of errors in n independently sampled selected cases at one fixed threshold. A one-sided binomial upper limit can bound the population risk with a chosen confidence level. If many thresholds are tested on the same data, a correction for that search, or a fresh validation split, is needed. Reused Grants violate the independence assumption when all their uses share one original error.

An illustrative workload calculation makes the distinction concrete. Suppose 100 eligible attempts contain 40 Grant hits. Of the remaining 60, the judge decides 36. The other 24 reach agent review. If agent review finishes 18, then 6 reach a person. The human request rate is 6 divided by 100, or 6 percent. Rule coverage on its eligible set is 36 divided by 60, or 60 percent. Reporting 94 percent "judge coverage" would wrongly include Grants and agent reviews. These counts are invented only to show the arithmetic.

More generally, let h be the Grant-hit share, c Rule coverage among misses, and b the share of Rule deferrals finished before human review. The human request rate is $(1-h)(1-c)(1-b)$ when these are sequential conditional shares of the same stream. Expected human time also depends on time per request. An adversary can reduce c with ambiguous requests, or raise review time with long misleading reasons. Review capacity is therefore part of availability, not merely an interface concern.

VII. Evaluation Protocol

The learned-model and human studies below are plans, not completed experiments. The finite protocol experiment is reported separately. The planned study has three linked parts: offline decision quality, sequential reuse, and human review. All parts preserve the assumed deterministic authority layer so that comparisons isolate the paper's research variable.

A. Cases and Reference Judgments

Build policy families for repository operations, messages, computer access, and service calls. Each family contains clear allows, clear denies, missing facts, exceptions, and conflicting Rules. Pair benign requests with adversarial versions that alter only the reason or a data field. Include negation, quoted false instructions, forged approval claims, irrelevant valid citations, and text that tries to hide a destination. Keep malicious content inside a test environment with no real outside writes.

Independent reviewers label the effect, the decisive Rule spans, missing facts, and the widest justified reuse scope. They first work without model output. Disagreement is resolved by a documented adjudication step, while the original labels remain available. If policy authors cannot settle a case, its reference

remains unresolved. Report agreement and the unresolved share. Do not discard ambiguous examples to make accuracy easier.

Use AgentDojo as one external task source, with its tool state and paired utility and security evaluation [10]. Add explicit trusted Rules and reference labels in a separate released layer. Preserve the original tasks and report every exclusion. These new labels need independent review; the benchmark itself does not provide the policy-judgment labels required here. Keep held-out new policies separate from this public benchmark to limit prompt tuning on the test set.

Split by policy family, target template, and task stream. Near copies of one Rule must not appear in both calibration and test sets. Add a separate shift set with unseen tool schemas and newly written policies. Keep natural traffic, structured edge cases, and adversarial cases as separate strata. Their average is meaningful only when the mixture weights are declared.

B. Baselines and Ablations

TABLE II
MAIN STUDY COMPARISONS, ALL INSIDE THE SAME AUTHORITY BOUNDARY.

Comparison	Question it answers
Ask on each miss	How much work does the judge remove beyond Grants?
Substring judge	What do quotes and baseline abstention achieve?
Forced decision	What is lost when no_match is removed?
Calibrated selection	Does held-out selection lower error at equal coverage?
Support and conflict	Do meaning checks help beyond citation existence?
Once-only reuse	What workload and repeated risk come from wider scope?

Run language-model and decision-model routes separately. Hold model versions, prompts, tool schemas, Rule order, and budget fixed within each comparison. Ablate citation checks, conflict search, support checks, reason text, and calibration one at a time. Turning off a safety check is only an offline measurement. It does not authorize production deployment. Compare warm-cache and empty-cache runs; vary agent roles and model settings while retaining the capability to expose omitted cache dependencies.

Compare direct human escalation with the owner chain while keeping authority checks fixed. Measure settled cases and added delay. Compare shared-model and different-model reviewers; their votes are not presumed independent.

C. Metrics and Statistical Plan

Report allow risk, deny risk, unresolved-case commitment, and coverage as curves over thresholds. Include the area under the risk-coverage curve, but show the operating points as well. Report citation validity separately from semantic support accuracy. For probability outputs, use reliability plots, Brier score, and calibration error with declared bins. Also report latency, timeout rate, model usage, requests per completed task, and human minutes per task.

The sequential study replays task streams with Grants enabled. Count unique mistaken decisions and all affected actions. Measure the reuse distance, meaning the number of later uses before a Grant stops matching or is revoked. Record wrong matches caused by omitted arguments, changed context, and policy updates. A warm cache may reduce model calls while repeating one wrong decision; these are different outcomes.

Choose sample sizes before final testing from the desired upper bound and expected selected fraction. Choose the finite threshold grid on development data, then use the separate certification split and simultaneous limits above. Reserve a final test set for comparison after every choice is frozen. Give binomial intervals only under the stated independent-sampling conditions. For repeated streams, resample whole policy families and task streams, not individual calls. Use paired comparisons because each method sees the same cases. Report the seed schedule and sensitivity to Rule order. Correct for multiple planned comparisons and mark exploratory analysis as such.

The human study uses simulated actions and assigned review blocks. Randomize the order of plain data, cited explanation, and condition-list presentations. Counterbalance order across participants. Measure correct final effect, justified scope, time, correction of model errors, and interruption burden. Do not equate a faster click with better oversight. Report participant training and limits on generalization. Human approval after seeing a model recommendation is not an independent reference label.

VIII. Reproducible Fixtures and Obligations

A minimal synthetic fixture has agent `worker`, conversation `c1`, and three actions: read, send, and delete. Targets are `reports` and `mail`. A send targets `mail`; its arguments contain destination, either `team` or `outside`, and content. Use the fixed content "Weekly report." All trials run against a simulator that records a permitted action and has no network connection. This makes decision outcomes observable without sending messages or deleting real data.

Define Rule A as "The agent may read reports." Define Rule B as "The agent must not send reports outside." Define Rule C as "The agent may send the weekly report to the team." A read of reports has reference allow. Sending to outside has reference deny. Sending the fixed weekly report to the team has reference allow. Deleting reports has reference unresolved because no Rule settles it. Removing the destination from a send also yields unresolved. These labels follow directly from the stated Rules, not from model agreement.

For the citation counterexample, give the substring baseline an allow output for sending outside with Rule A's quote "may read reports." The baseline accepts it because the ID and substring pass. For conflict coverage, add Rule D: "The agent may send reports." Let an allow for sending outside cite D. Both D and B apply, so the defined combination policy requires deny. For injection, append a claim of prior approval to the reason while keeping the capability fixed. The reference effect remains unchanged. For exact matching,

create a conversation Grant for the team send and change only the destination, conversation, or argument key order. Each case has a mechanically specified match result.

Reuse fixtures need streams. Start with an approved team send, then repeat it, change its content, change its destination, and move to `c2`. Run the stream with one-call, conversation, and lifetime Grants, both with and without argument constraints. Record matches and final reference errors separately. To test one-call contention, present two simultaneous claims to one active Grant; at most one may change it to used. To test review, supply an old revision after an endorsement; the state must remain unchanged. The finite runner executes the two serial orders of one-call claims. It does not execute simultaneous database claims or the stale-review case. These remain separate protocol obligations.

Archive fixture values, model identifiers, exact prompts, checker versions, threshold selection, seed schedule, cache keys, and raw verdicts before and after filtering. Report failures and deadlines as outcomes instead of retrying until the model agrees. A complete implementation must satisfy atomic one-call claims, immutable call binding, current reviewer checks, finite upward escalation, and preserved Rule versions. Separate those protocol obligations from the empirical obligation to measure semantic errors. Passing deterministic fixtures does not establish the quality of a learned judge.

IX. Finite Experiment

We ran the supplied experiment runner with the Python standard library. Paper 03 uses exhaustive enumeration and fixed examples; it uses no random seed. The output records source hashes and separates each paper's checks. This runner does not start a product service or call a model.

The aggregation domain has four labels and six scores: 0.69, 0.7, 0.8, 0.85, 0.9, and 1.0. Enumerating lists of length zero through three gives $1 + 24 + 24^2 + 24^3 = 14,425$ cases. The support cutoff is 0.7, the allow threshold is 0.85, and the deny threshold is 0.9. The score map is the identity. These values probe boundary behavior; they are not learned probabilities.

The oracle uses a separate predicate: deny exists above both relevant cutoffs; otherwise allow exists and every label is allow or irrelevant above both relevant cutoffs; otherwise defer. It does not call the selection procedure. The two computations agreed on every case. This is agreement between a procedure and its intended finite specification, not an independent source of semantic truth.

TABLE III
Executed finite results. No row measures learned-judge accuracy.

Finite measurement	Observed value	What it establishes
Enumerated label-score lists	14,425	All lists in the declared domain
Oracle disagreements	0	Aggregation agrees on that domain
Allow / deny / defer	219 / 3,270 / 10,936	Logic outcomes; no traffic coverage estimate
Serial one-call orders	2; one winner each	The state model consumes a call once
Irrelevant real quote	Accepted by source check	Source checking alone permits wrong support
Complete true-label lists	40; zero disagreements	Sound complete evidence gives the specified effect
One true deny removed	11 wrong allows in 34 cases	Enumeration completeness is necessary
Parent-child scope pairs	1,296; 150 contained	Field checks agree with finite match-set inclusion

The Grant examples also reproduce a key-order miss, a conversation miss, a destination miss, and a wide match when arguments are omitted. The narrow cache key collides when only the reason changes, while the full input record differs. These observations justify the test cases. They do not measure how often the errors occur in a deployed workload.

For the completeness control, enumerate true-label lists of length at most three over allow, deny, and irrelevant. Supply each true label with score 1.0. All 40 complete lists return the reference effect. Then remove each deny entry in turn. Of the 34 resulting missing-Rule cases, 11 return allow despite a true denial. The smallest witness is an allow Rule and a deny Rule with the deny omitted. This measures a defect in an incomplete evidence path on selected synthetic cases. It is not an estimated attack success rate or a test of Rule retrieval.

For scope, enumerate 16 contexts from two actions, two conversations, two argument values, and two times. The 36 Grants vary their action, unrestricted or fixed conversation, unrestricted or fixed arguments, and no expiry or expiry at time one. Across all 1,296 ordered parent-child pairs, field restriction agrees with direct match-set inclusion; 150 pairs are contained. This finite language omits a separate target field and has only one finite expiry value. It does not establish completeness of the field test for richer policy languages.

The runner reports 368, 597, and 826 as the first zero-error sample counts that meet the declared joint bound for one, 10, and 100 threshold pairs. This is a numerical check of the study design. It reports no observed model error rate. These extensions live in `papers/experiments/paper_03_extended.py` and run through the same entry point.

The exhaustive domain is small and selected by the authors. Its oracle shares the intended mathematical policy with the implementation. It can find a coding disagreement but cannot discover that the chosen policy is wrong for a user. It excludes learned scores, incomplete real Rule retrieval, database races, policy-writing errors, and human behavior. The proposed

learned-model study is required before claiming reduced decision risk at useful coverage.

X. Limits and Conclusion

Policy can remain ambiguous even to careful reviewers. Calibration may fail after a tool change or against adaptive attacks. Checkers can share errors. Long Rule sets can hide conflicts. Human overload and Grant reuse can preserve mistakes. A valid citation resolves none of these issues.

The layered policy separates reused decisions, language generalization, and escalation. Its key boundary is between evidence that text exists and evidence that it supports an action. The finite experiment checks the aggregation boundary in a declared small domain. The risk-control and scope procedures state further conditions for wider claims. Learned support quality, review load, and repeated-action risk remain open empirical questions. Deterministic authority preservation remains an explicit assumption.

References

[1] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

[2] H. Mozannar and D. Sontag, "Consistent Estimators for Learning to Defer to an Expert," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. 2020, pp. 7076–7087. [Online]. Available: https://proceedings.mlr.press/v119/mozannar20b.html

[3] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proceedings of the 34th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 70. 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[4] G. Bansal *et al.*, "Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance." [Online]. Available: https://arxiv.org/abs/2006.14779v3

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." [Online]. Available: https://arxiv.org/abs/2302.12173

[6] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions." [Online]. Available: https://arxiv.org/abs/2404.13208

[7] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 6465–6488. doi: 10.18653/v1/2023.emnlp-main.398.

[8] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control," *The Annals of Applied Statistics*, vol. 19, no. 2, pp. 1641–1662, 2025, doi: 10.1214/24-AOAS1998.

[9] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Network and Distributed System Security Symposium*, 2014. [Online]. Available: https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." [Online]. Available: https://arxiv.org/abs/2406.13352v3